

SMART IoT AUTOMATION SYSTEM

Smart Motion Security & Monitoring System

Objective

The objective of this task is to upgrade the basic motion detection system developed in Task 2 into a smart automation system. The project demonstrates how an embedded system can automatically make decisions based on sensor input, control output devices, monitor events, and provide IoT-based monitoring using Wi-Fi.

Introduction

The Smart Motion Security and Monitoring System is an IoT-based security project developed using an ESP32, PIR motion sensor, LED, and buzzer. The system detects movement in a particular area and automatically activates an alarm when motion is detected.

The ESP32 processes the signal received from the PIR sensor and controls the LED and buzzer according to the programmed conditions. The system also connects to Wi-Fi and provides a simple web page for monitoring security status.

Components Used

The components used in this project are:

- ESP32 development board
- PIR motion sensor
- LED
- 220 Ω resistor
- Buzzer
- Jumper wires
- Wokwi online simulator

Sensor Used

A PIR (Passive Infrared) motion sensor is used to detect movement. It works by detecting changes in infrared radiation produced by moving objects, particularly humans.

When motion is detected, the PIR sensor provides a HIGH digital signal to the ESP32. When there is no motion, it provides a LOW signal.

Circuit Connections

The PIR sensor VCC pin is connected to the 5V/VIN supply of the ESP32, and its GND pin is connected to GND. The output pin of the PIR sensor is connected to GPIO 27.

The LED is connected to GPIO 26 through a 220 Ω resistor. The cathode of the LED is connected to the GND.

The positive terminal of the buzzer is connected to GPIO 25, while the negative terminal is connected to GND.

Component	Connection
PIR VCC (+)	ESP32 VIN / 5V
PIR OUT (D)	GPIO 27
PIR GND (-)	GND
LED Anode (+)	GPIO 26 through 220 Ω resistor
LED Cathode (-)	GND
Buzzer (+)	GPIO 25
Buzzer (-)	GND

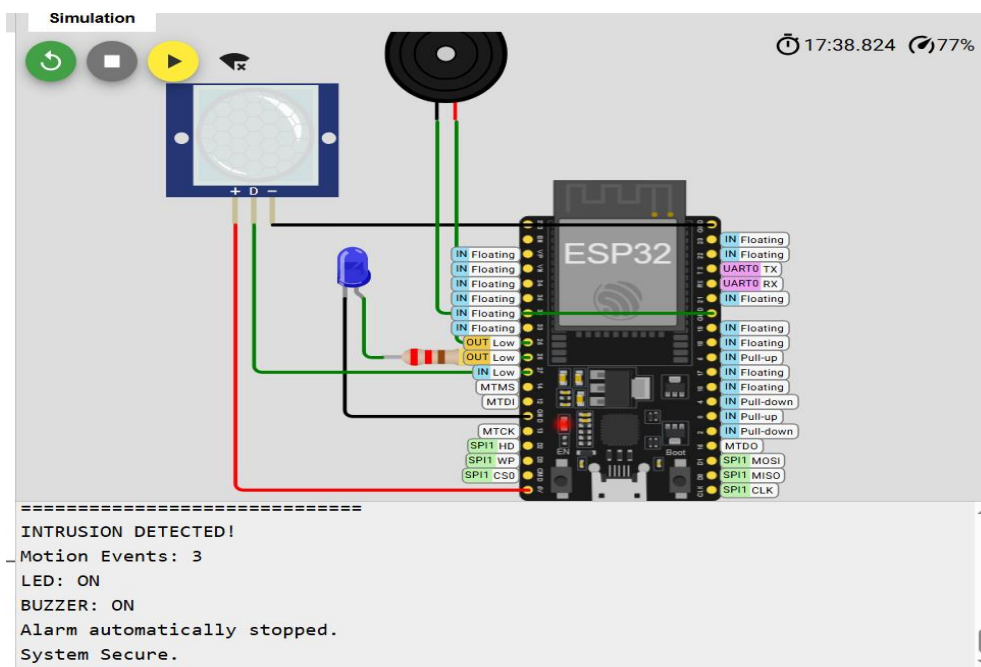
Working of the System

When the system starts, the ESP32 initializes the PIR sensor, LED, and buzzer and establishes a Wi-Fi connection.

The PIR sensor continuously monitors the surrounding area for movement. When a new motion event is detected, the ESP32 identifies the event and automatically activates the LED and buzzer.

The system also increases the motion-event count and displays the information on the Serial Monitor. The current security status and number of detected motion events are also made available through the ESP32 web server.

The alarm remains active for five seconds. After five seconds, the ESP32 automatically switches OFF the LED and buzzer and returns the system to the secure state.



Automation Logic

The system uses IF/ELSE and conditional logic to make automatic decisions.

When motion is detected, the LED and buzzer are turned ON and the motion-event counter is increased.

When the alarm has been active for five seconds, the ESP32 automatically turns OFF the LED and buzzer.

When there is no motion and the alarm is not active, the system remains in a normal secure state.

IoT Extension

The ESP32 provides Wi-Fi connectivity to the project. In the Wokwi simulation, the ESP32 connects to the Wokwi-GUEST Wi-Fi network.

An HTTP web server is created on the ESP32. The web page provides information about the current security condition of the system.

The web interface displays the security status, number of motion events, and alarm conditions. This provides basic IoT-style monitoring without requiring a separate cloud platform.

Source Code

```
#include <WiFi.h>
#include <WebServer.h>

const char* ssid = "Wokwi-GUEST";
const char* password = "";

WebServer server(80);

const int PIR_PIN = 27;
const int LED_PIN = 26;
const int BUZZER_PIN = 25;

int motionCount = 0;
int previousMotion = LOW;

bool alarmActive = false;
unsigned long alarmStartTime = 0;
```

```
void handleRoot() {  
  
    int motion = digitalRead(PIR_PIN);  
  
    String status;  
    String alarm;  
  
    if (motion == HIGH) {  
        status = "INTRUSION DETECTED!";  
        alarm = "ON";  
    }  
    else {  
        status = "SYSTEM SECURE";  
        alarm = "OFF";  
    }  
  
    String html = "<!DOCTYPE html>";  
    html += "<html>";  
    html += "<head>";  
    html += "<meta http-equiv='refresh' content='2'>";  
    html += "<title>Smart Security System</title>";  
    html += "</head>";  
  
    html += "<body>";  
    html += "<h1>SMART SECURITY SYSTEM</h1>";  
  
    html += "<h2>Status: " + status + "</h2>";  
    html += "<p>Motion Events: " + String(motionCount) + "</p>";  
    html += "<p>Alarm: " + alarm + "</p>";  
  
    html += "</body>";  
    html += "</html>";  
    server.send(200, "text/html", html);  
}
```

```
void setup() {  
  
    Serial.begin(115200);  
  
    pinMode(PIR_PIN, INPUT);  
    pinMode(LED_PIN, OUTPUT);  
    pinMode(BUZZER_PIN, OUTPUT);  
  
    digitalWrite(LED_PIN, LOW);  
    digitalWrite(BUZZER_PIN, LOW);  
  
    Serial.println("Connecting to WiFi...");  
  
    WiFi.begin(ssid, password);  
  
    while (WiFi.status() != WL_CONNECTED) {  
        delay(500);  
        Serial.print(".");  
    }  
  
    Serial.println();  
    Serial.println("WiFi Connected!");  
  
    Serial.print("IP Address: ");  
    Serial.println(WiFi.localIP());  
  
    server.on("/", handleRoot);  
  
    server.begin();  
  
    Serial.println("Web Server Started");  
}  
  
void loop() {  
  
    server.handleClient();  

```

```
int motion = digitalRead(PIR_PIN);

if (motion == HIGH && previousMotion == LOW) {

    motionCount++;

    alarmActive = true;
    alarmStartTime = millis();

    digitalWrite(LED_PIN, HIGH);
    digitalWrite(BUZZER_PIN, HIGH);

    Serial.println("=====");
    Serial.println("INTRUSION DETECTED!");

    Serial.print("Motion Events: ");
    Serial.println(motionCount);

    Serial.println("LED: ON");
    Serial.println("BUZZER: ON");
}

if (alarmActive && millis() - alarmStartTime >= 5000) {

    alarmActive = false;

    digitalWrite(LED_PIN, LOW);
    digitalWrite(BUZZER_PIN, LOW);

    Serial.println("Alarm automatically stopped.");
    Serial.println("System Secure.");
}

previousMotion = motion;
```

```
    delay(50);  
}
```

Monitoring

The Serial Monitor is used to observe the system's operation in real time.

When the system starts, messages such as Wi-Fi connection status, IP address, and web server status are displayed.

When motion is detected, the Serial Monitor displays an intrusion warning, motion-event count, LED status, and buzzer status.

After the five-second alarm period, it displays that the alarm has automatically stopped and that the system is secure.

When the ESP32 starts:

```
Connecting to WiFi...  
WiFi Connected!  
IP Address: <ESP32 IP address>  
Web Server Started
```

When motion is detected:

```
INTRUSION DETECTED!  
Motion Events: 1  
LED: ON  
BUZZER: ON
```

After five seconds:

```
Alarm automatically stopped.  
System Secure.
```

Expected Output

When there is no motion, the system remains secure, the LED is OFF, and the buzzer is OFF.

When motion is detected, the system displays an intrusion alert, turns ON the LED, and activates the buzzer.

The web monitoring page also changes its status according to the detected condition.

Testing and Results

The system was tested under different conditions. When there was no motion, the LED and buzzer remained OFF, and the system showed the secure status.

When motion was detected, the LED and buzzer were automatically activated, and an intrusion message was displayed.

The motion-event counter increased whenever a new motion event occurred. The alarm automatically switched OFF after five seconds.

The Wi-Fi connection and web server were also tested successfully for monitoring the security status.

Advantages

The system provides automatic motion detection and alarm activation. It allows real-time monitoring through the Serial Monitor and web interface. The motion-event counter provides basic event logging, while Wi-Fi connectivity adds IoT functionality.

The system is simple, low-cost, and can be expanded for real-world security applications.

Applications

This system can be used for home security, office monitoring, laboratory security, restricted-area monitoring, smart buildings, and other IoT-based security applications.

Future Enhancements

The project can be further improved by connecting it to cloud platforms such as ThingSpeak or Blynk. Mobile notifications can also be added to alert users when motion is detected.

A camera can be integrated for visual verification of an intrusion. An LCD or OLED display can be added for local monitoring. Date and time information can also be stored to maintain a complete history of security events.

Conclusion

The Smart Motion Security and Monitoring System successfully upgrades the basic motion detection project into a smart automated IoT system. The ESP32 reads the PIR sensor, processes the sensor information, makes decisions using programmed logic, and automatically controls the LED and buzzer.

The addition of Wi-Fi and an HTTP web server allows the system status and motion events to be monitored digitally. This project provided practical knowledge of sensor interfacing, embedded programming, automation, real-time monitoring, and basic IoT communication